

10.6.1 Assembling more complex monoids

Some data structures form interesting monoids as long as the types of the elements they contain also form monoids. For instance, there's a monoid for merging key-value Maps, as long as the value type is a monoid.

Listing 10.1 Merging key-value Maps

```
def mapMergeMonoid[K,V](V: Monoid[V]): Monoid[Map[K, V]] =
  new Monoid[Map[K, V]] {
    def zero = Map[K,V]()
    def op(a: Map[K, V], b: Map[K, V]) =
      (a.keySet ++ b.keySet).foldLeft(zero) { (acc,k) =>
        acc.updated(k, V.op(a.getOrElse(k, V.zero),
                           b.getOrElse(k, V.zero)))
      }
  }
```

Using this simple combinator, we can assemble more complex monoids fairly easily:

```
scala> val M: Monoid[Map[String, Map[String, Int]]] =
  | mapMergeMonoid(mapMergeMonoid(intAddition))
M: Monoid[Map[String, Map[String, Int]]] = $anon$1@21dfac82
```

This allows us to combine nested expressions using the monoid, with no additional programming:

```
scala> val m1 = Map("o1" -> Map("i1" -> 1, "i2" -> 2))
m1: Map[String,Map[String,Int]] = Map(o1 -> Map(i1 -> 1, i2 -> 2))

scala> val m2 = Map("o1" -> Map("i2" -> 3))
m2: Map[String,Map[String,Int]] = Map(o1 -> Map(i2 -> 3))

scala> val m3 = M.op(m1, m2)
m3: Map[String,Map[String,Int]] = Map(o1 -> Map(i1 -> 1, i2 -> 5))
```



EXERCISE 10.17

Write a monoid instance for functions whose results are monoids.

```
def functionMonoid[A,B](B: Monoid[B]): Monoid[A => B]
```



EXERCISE 10.18

A bag is like a set, except that it's represented by a map that contains one entry per element with that element as the key, and the value under that key is the number of times the element appears in the bag. For example:

```
scala> bag(Vector("a", "rose", "is", "a", "rose"))
res0: Map[String,Int] = Map(a -> 2, rose -> 2, is -> 1)
```